



5장 합성곱 신경망 1 (5.3~)

4팀 윤정 진예원 조수진

목차

#01 전이 학습 (1)

#02 전이 학습 (2)

#03 설명 가능한 CNN, 그래프 합성곱 네트워크



전이 학습 (1)



#01 전이 학습 (1)

#1 전이 학습의 개념과 필요성

#2 전이 학습 방법 : 특성 추출 기법

#3 특성 추출 기법 코드 설명

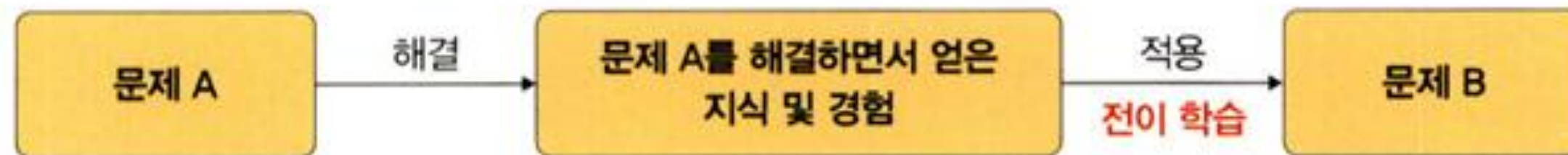
#01 전이 학습의 개념과 필요성

합성곱 신경망 기반의 딥러닝 모델 : 많은 양의 데이터가 필요
하지만 큰 데이터셋을 얻는 것은 어려움 (많은 돈과 시간 필요)

➡ 전이 학습 (transfer learning)

이미지넷처럼 큰 데이터셋을 써서 훈련된 모델의 가중치를 가져와
해결하려는 과제에 맞게 보정해서 사용하는 방법

비교적 적은 수의 데이터로 원하는 과제 해결 가능



큰 데이터셋을 사용하여 훈련된 모델 = 사전 훈련된 모델 (네트워크)

#02 전이 학습 방법 : 특성 추출 기법

사전 훈련된 모델의 마지막에 완전연결층을 새로 만듦

- ➡ 마지막 완전연결층만 학습, 나머지 계층은 학습X
- 마지막 완전연결층 : 이미지의 카테고리를 결정

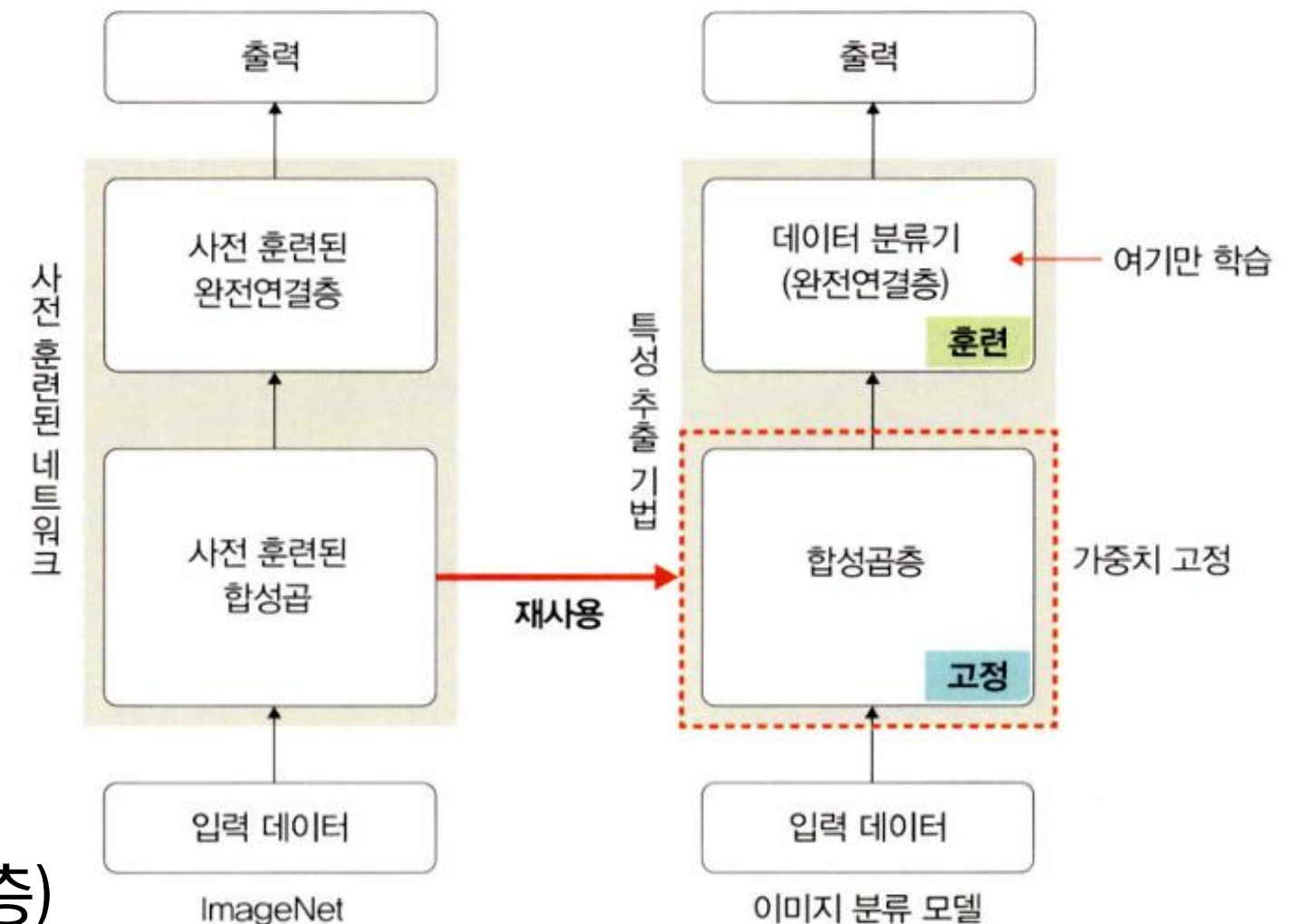
이미지 분류를 위해 두 부분으로 구성

- 합성곱층 : 합성곱층과 풀링층으로 구성
- 데이터 분류기 (완전연결층) : 추출된 특성을 입력 받아 이미지에 대한 클래스 분류

사전 훈련된 모델의 합성곱층 ➡ 새로 만든 데이터 분류기 (완전연결층)

사전 훈련된 모델 (이미지 분류 모델)

- Xception, Inception V3, ResNet50, VGG16, VGG19, MobileNet



#02 전이 학습 방법 : 특성 추출 기법

사전 훈련된 모델 (이미지 분류 모델)

- ImageNet과 같은 대규모 데이터로 미리 학습해 둔 CNN 모델
 - ➡ 일반적인 특징을 이미 학습, 적은 데이터로도 활용 가능, 학습 시간 단축

Xception	Depthwise Separable Convolution을 사용하여 계산량을 줄이면서 효율적으로 특징 추출
Inception V3	다양한 크기의 필터를 사용해 여러 크기의 특징을 효과적으로 추출
ResNet50	Skip Connection을 사용해서 매우 깊은 네트워크에서도 학습이 가능하게 함
VGG16	작은 필터를 여러 번 쌓은 구조
VGG19	VGG16보다 층이 더 많아서 더 복잡한 특징을 추출
MobileNet	계산량과 파라미터를 크게 줄여 가볍고 빠르게 특징 추출

#03 특성 추출 기법 코드 설명

#라이브러리 호출

```
import os
import time
import copy
import glob
import cv2 ----- 앞에서 설치한 OpenCV 라이브러리
import shutil
```

```
import torch
import torchvision ----- 컴퓨터 비전(computer vision) 용도의 패키지
import torchvision.transforms as transforms ----- 데이터 전처리를 위해 사용되는 패키지
import torchvision.models as models ----- 다양한 파이토치 네트워크를 사용할 수 있도록 도와주는 패키지
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
```

```
import matplotlib.pyplot as plt
```

OpenCV = Open Source Computer Vision Library

#03 특성 추출 기법 코드 설명

#이미지 데이터 전처리 방법 정의

```
data_path = '../chap05/data/catanddog/train' ----- 이미지 데이터가 위치한 경로 지정
```

```
transform = transforms.Compose(  
    [  
        transforms.Resize([256, 256]),  
        transforms.RandomResizedCrop(224),  
        transforms.RandomHorizontalFlip(),  
        transforms.ToTensor()  
    ]  
) ----- ①
```

```
train_dataset = torchvision.datasets.ImageFolder(  
    data_path,
```

```
        transform=transform  
) ----- ②  
train_loader = torch.utils.data.DataLoader(  
    train_dataset,  
    batch_size=32,  
    num_workers=8,  
    shuffle=True  
) ----- ③
```

```
print(len(train_dataset))
```

Torchvision.transform : 이미지 데이터를 모델의 입력으로 변환

- Resize : 이미지 크기 조정
- RandomResizedCrop : 이미지를 랜덤한 크기, 비율로 자름
- RandomHorizontalFlip : 이미지를 랜덤하게 수평으로 뒤집음
- ToTensor : 이미지 데이터를 텐서로 변환

#03 특성 추출 기법 코드 설명

#RandomResizedCrop

```
def show_images(imgs, num_rows, num_cols, scale=2):
    aspect_ratio = imgs[0].shape[0]/imgs[0].shape[1] ----- 확장할 이미지의 크기 조정
    figsize = (num_cols * scale, num_rows * scale * aspect_ratio)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    for i in range(num_rows):
        for j in range(num_cols):
            axes[i][j].imshow(imgs[i * num_cols + j].asnumpy())
            axes[i][j].axes.get_xaxis().set_visible(False) ----- x축 전체를 숨김
            axes[i][j].axes.get_yaxis().set_visible(False) ----- y축 전체를 숨김
    plt.subplots_adjust(hspace=0.1, wspace=0)
    return axes

def apply(img, aug, num_rows=2, num_cols=4, scale=3):
    Y = [aug(img) for _ in range(num_rows * num_cols)] ----- 다양한 샘플을 얻기 위해 여러 번
    show_images(Y, num_rows, num_cols, scale) ----- 데이터 확장 적용

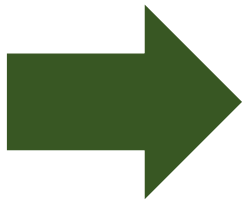
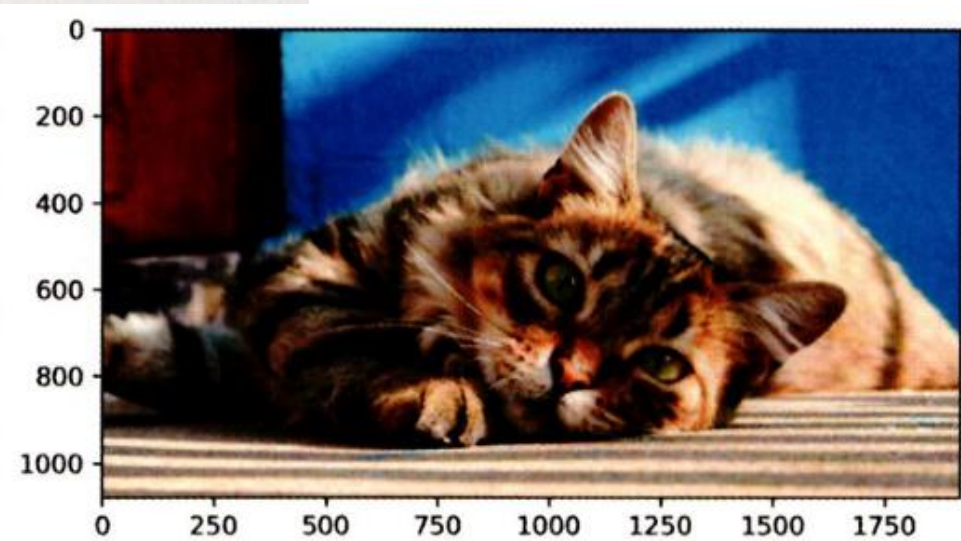
shape_aug = transforms.RandomResizedCrop(size=(200, 200),
                                          scale=(0.1, 1),
                                          ratio=(0.5, 2))

apply(example_image, shape_aug)
```

Scale : 얼마나 크게 자를지 결정

Ratio : 잘라낼 영역의 가로, 세로 비율 결정

Size : 잘라낸 영역의 최종적인 사이즈 결정



#03 특성 추출 기법 코드 설명

#이미지 데이터 전처리 방법 정의

data_path = '../chap05/data/catanddog/train' ----- 이미지 데이터가 위치한 경로 지정

```
transform = transforms.Compose([
    transforms.Resize([256, 256]),
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor()
]) ----- ①
```

```
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
) ----- ②
```

```
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
) ----- ③
```

print(len(train_dataset)) ➡ 385

Datasets.ImageFolder : 데이터로더가 불러올 대상과 방법 정의

- 첫 번째 파라미터 : 불러올 데이터의 위치, 경로
- Transform : 이미지 데이터에 대한 전처리

정의한 ImageFolder를 데이터로더에 할당

- 첫 번째 파라미터 : 데이터셋 지정
- Batch_size : 배치 크기 (한 번에 불러올 데이터양) 설정
- Num_workers : 하위 프로세스 개수 결정
- Shuffle : 데이터를 무작위로 섞을지 결정

#03 특성 추출 기법 코드 설명

#학습에 사용될 이미지 출력

```
samples, labels = iter(train_loader).next() ----- ①
classes = {0:'cat', 1:'dog'} ----- 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): ----- 24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) ----- 레이블 정보(클래스)를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0))) ----- ②
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

24개의 이미지에 대해 레이블 정보와 함께 출력

Train_loader에서 데이터를 하나씩 꺼내옴

- Iter() : 전달된 데이터의 반복자를 꺼내 반환
- Next() : 반복자가 다음에 출력해야 할 요소를 반환

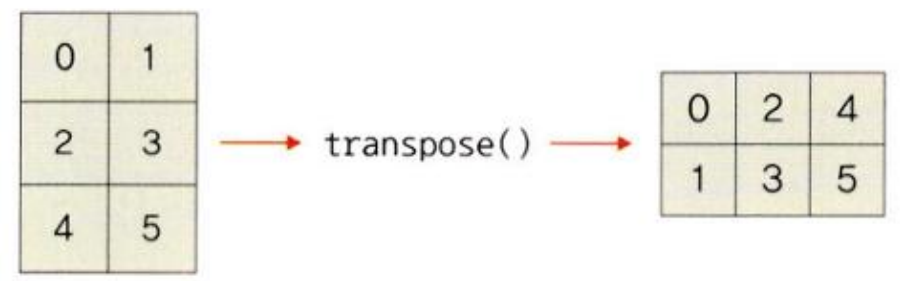
Train_loader에서 samples, labels 값을
순차적으로 꺼내서 저장

#03 특성 추출 기법 코드 설명

#학습에 사용될 이미지 출력

```
samples, labels = iter(train_loader).next() ----- ①
classes = {0:'cat', 1:'dog'} ----- 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): ----- 24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) ----- 레이블 정보(클래스)를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0))) ----- ②
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

Np.transpose : 행과 열을 바꾸어 행렬의 차원을 변환



행렬의 내적 연산을 위해 변환
(연산할 행렬의 행과 열의 수가 동일해야 함)

24개의 이미지에 대해 레이블 정보와 함께 출력

Torch.Size([32, 3, 224, 224]) 그대로 진행 시 오류 발생

➡ Np.transpose를 사용해서 (224, 224, 3) 형태로 변환

#03 특성 추출 기법 코드 설명

#학습에 사용될 이미지 출력

```
samples, labels = iter(train_loader).next() ----- ①
classes = {0:'cat', 1:'dog'} ----- 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): ----- 24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) ----- 레이블 정보(클래스)를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0))) ----- ②
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

24개의 이미지에 대해 레이블 정보와 함께 출력

Np.transpose : 행과 열을 바꾸어 행렬의 차원을 변환

0	1
2	3

→ transpose() →

0	2	4
---	---	---

> np.transpose(exam, (2, 1, 0)) ----- (2, 3, 4) 크기를 (4, 3, 2)로 변환

array([[[0, 12],
[4, 16],
[8, 20]],

[[1, 13],
[5, 17],
[9, 21]],

[[2, 14],
[6, 18],
[10, 22]],

[[3, 15],
[7, 19],
[11, 23]])

행렬C
(연산)

Torcl

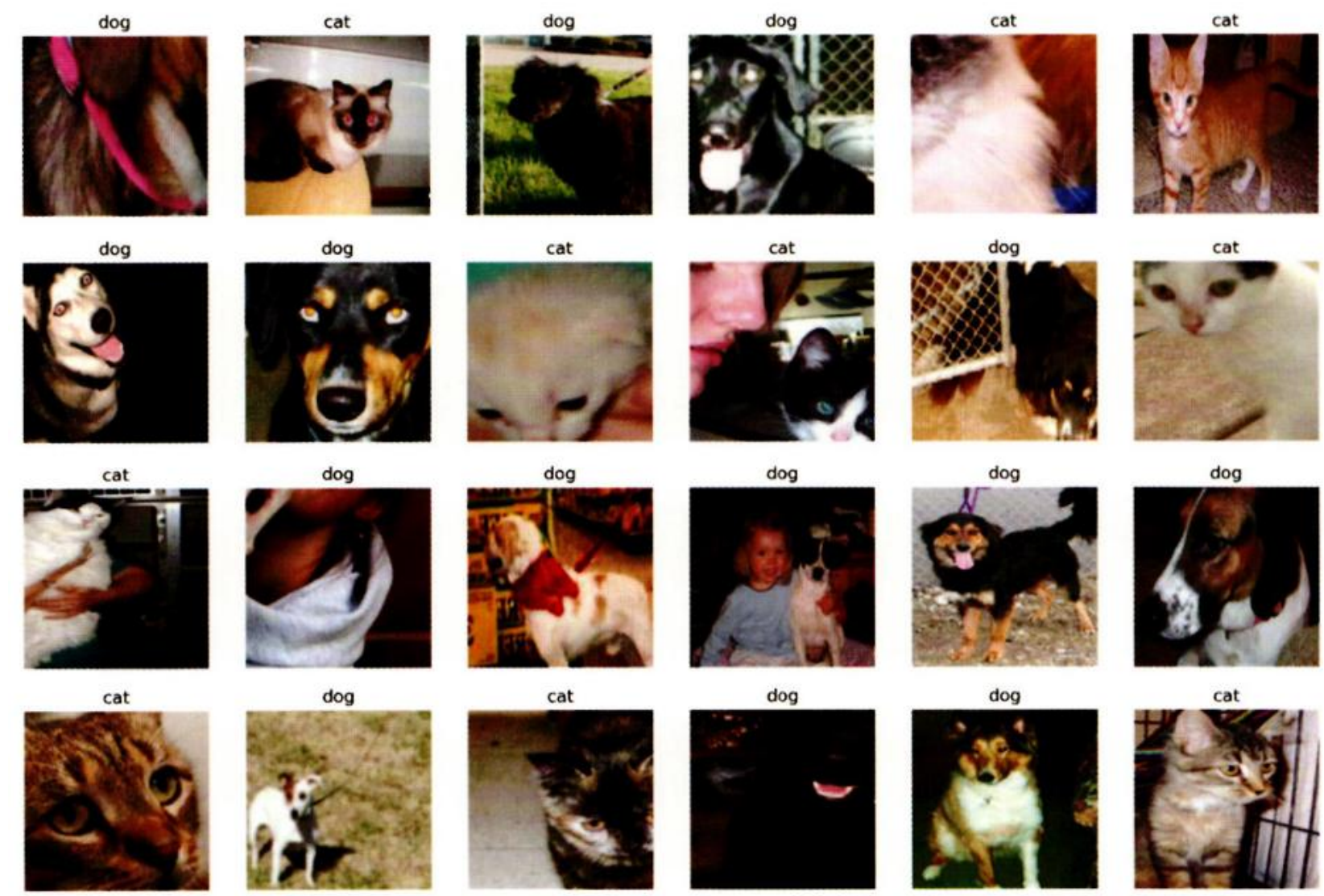


2류 발생

형태로 변환

#03 특성 추출 기법 코드 설명

#24개의 이미지 출력 결과



#03 특성 추출 기법 코드 설명

#사전 훈련된 모델 내려 받기

```
resnet18 = models.resnet18(pretrained=True) ----- pretrained=True는 사전 학습된 가중치를  
사용하겠다는 의미
```

ResNet18

- 이미지 분류에 사용하는 CNN 모델 중 하나
- 50개의 계층으로 구성된 합성곱 신경망
- ImageNet 데이터베이스의 영상을 이용하여 훈련된 신경망
- 입력 제약이 매우 크고 충분한 메모리가 없으면 학습 속도가 느림

#03 특성 추출 기법 코드 설명

#사전 훈련된 모델

```
import torchvision.models as models
resnet18 = models.resnet18()
alexnet = models.alexnet()
vgg16 = models.vgg16()
squeezenet = models.squeezenet1_0()
densenet = models.densenet161()
inception = models.inception_v3()
googlenet = models.googlenet()
shufflenet = models.shufflenet_v2_x1_0()
mobilenet_v2 = models.mobilenet_v2()
mobilenet_v3_large = models.mobilenet_v3_large()
mobilenet_v3_small = models.mobilenet_v3_small()
resnext50_32x4d = models.resnext50_32x4d()
wide_resnet50_2 = models.wide_resnet50_2()
mnasnet = models.mnasnet1_0()
```

무작위의 가중치로 모델 구성

```
import torchvision.models as models
resnet18 = models.resnet18(pretrained=True)
alexnet = models.alexnet(pretrained=True)
squeezenet = models.squeezenet1_0(pretrained=True)
vgg16 = models.vgg16(pretrained=True)
densenet = models.densenet161(pretrained=True)

inception = models.inception_v3(pretrained=True)
googlenet = models.googlenet(pretrained=True)
shufflenet = models.shufflenet_v2_x1_0(pretrained=True)
mobilenet_v2 = models.mobilenet_v2(pretrained=True)
mobilenet_v3_large = models.mobilenet_v3_large(pretrained=True)
mobilenet_v3_small = models.mobilenet_v3_small(pretrained=True)
resnext50_32x4d = models.resnext50_32x4d(pretrained=True)
wide_resnet50_2 = models.wide_resnet50_2(pretrained=True)
mnasnet = models.mnasnet1_0(pretrained=True)
```

사전 학습된 모델의 가중치 값 사용



5.3 전이 학습 (2)

특성 추출 실습 이어서 · 미세 조정 기법

#00 Part B 에서 다룰 내용

Part A 에서 데이터 준비 + ResNet18 로드까지 끝냈으니 , 이제 그 모델을 어떻게 학습시키고 평가하는지 이어서 봅니다 .



완료 (Part A)

이번 발표 (Part B)

5.3.1 특성 추출 실습 (이어서)

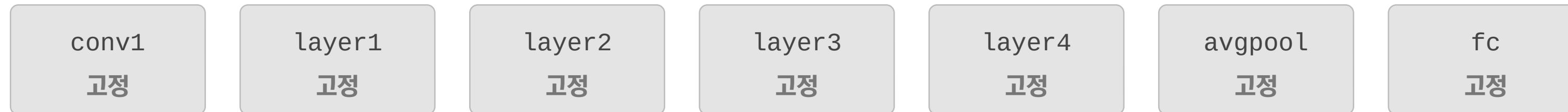


#01 파라미터 학습 유무 지정 (코드 5-16)

사전 훈련된 모델의 파라미터 잠그기

```
def set_parameter_requires_grad(model, feature_extracting=True):  
    if feature_extracting:  
        for param in model.parameters():  
            param.requires_grad = False    # ①  
  
set_parameter_requires_grad(resnet18)
```

resnet18.parameters() 전체



requires_grad = False → 역전파가 이 값들을 건드리지 않음

① requires_grad = False

- 역전파 중 이 파라미터에 대한 **변화 (기울기)** 를 **계산하지 않음**
- 모델의 **일부를 고정**하고 나머지만 학습할 때 사용
- 고정되는 부분 = **합성곱층 + 풀링층**
- Part A 에서 불러온 resnet18 의 **모든 파라미터**가 잠김

Tip requires_grad 는 " 이 파라미터 기울기 계산해 줄까 ?" 스위치 → **False 로 끄면 잠금**

#02 ResNet18 에 완전연결층 추가 (코드 5-17)

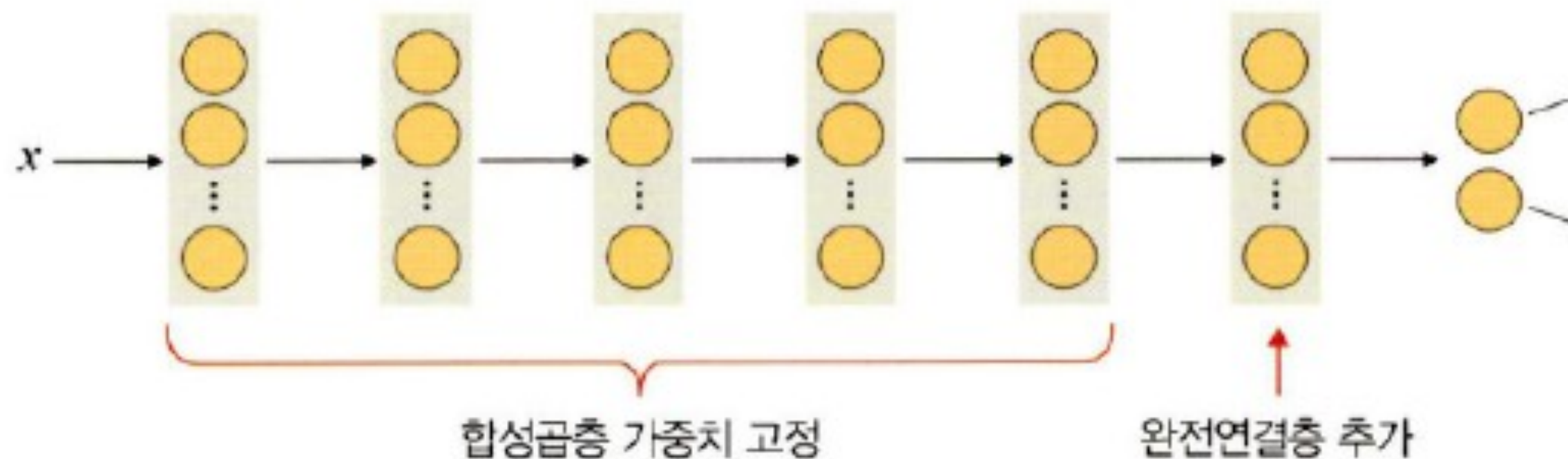
```
resnet18.fc = nn.Linear(512, 2)    # 2 classes
```

새로운 분류기 붙이기

- 내려받은 ResNet18 의 마지막 부분을 개 / 고양이 분류용 완전연결층으로 교체
- 입력 **512** : 마지막 합성곱 블록의 출력 채널 수
- 출력 **2** : cat, dog

Point 새로 만든 층은 기본값이 **requires_grad = True**
→ 앞에서 전부 잠갔어도
이 층만 학습됨

▼ 그림 5-36 ResNet18에 완전연결층 추가



#03 학습할 파라미터 확인 (코드 5-18)

```
for name, param in resnet18.named_parameters():  
    if param.requires_grad:  
        print(name, param.data)
```

출력 결과

```
fc.weight tensor([[ 0.0290, -0.0368, ..., 0.0337, 0.0388],  
                  [ 0.0173, -0.0082, ..., -0.0227, -0.0236]])  
fc.bias tensor([-0.0427, -0.0383])
```

- requires_grad = True 인 것만 출력 → **fc.weight, fc.bias** 두 개만 나옴
- 파라미터는 **weight** 와 **bias** 로 구성
- 5-16(잠금) + 5-17(fc 추가) 이 **의도대로 적용됐는지 검증**하는 단계

학습되는 파라미터 크기

fc.weight

(2, 512)

출력 2 개 × 입력 512 개

fc.bias

(2,)

출력 뉴런마다 1 개

#04 모델 객체 생성 및 손실 함수 정의 (코드 5-19)

```
model = models.resnet18(pretrained=True)

for param in model.parameters():
    param.requires_grad = False

model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters():
    param.requires_grad = True

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss()
print(model)
```

5-15 ~ 5-17 을 한 번에

5-15 사전 학습된 ResNet18 로드

5-16 전체 파라미터 고정 (합성곱층 잠금)

5-17 fc = Linear(512, 2) → 학습 허용

NEW 옵티마이저 · 손실 함수 정의

Point 옵티마이저에 **fc 파라미터만** 전달
→ 완전연결층만 업데이트
손실 함수 =
CrossEntropyLoss

#05 ResNet18 구조 한눈에 보기 (print(model))

블록	구성	출력 채널	크기 변화 (224 × 224 기준)
conv1	Conv2d(3, 64, kernel 7 × 7, stride 2, padding 3)	64	224 → 112
bn1 / relu	BatchNorm2d(64) / ReLU	64	–
maxpool	MaxPool2d(kernel 3, stride 2, padding 1)	64	112 → 56
layer1	BasicBlock × 2 (각 3 × 3 conv 2 개)	64	56 유지
layer2	BasicBlock × 2, 첫 블록 stride 2 + downsample	128	56 → 28
layer3	BasicBlock × 2, 첫 블록 stride 2 + downsample	256	28 → 14
layer4	BasicBlock × 2, 첫 블록 stride 2 + downsample	512	14 → 7
avgpool	AdaptiveAvgPool2d(output_size=(1, 1))	512	7 → 1
fc	Linear(in_features=512, out_features=2)	2	– (학습)

읽는 법

- **BasicBlock**
conv(3 × 3) → bn → relu → conv(3 × 3) → bn, 여기에 **입력을 더해 주는 잔차 연결**
- **downsample**
Conv2d(1 × 1, stride 2) + BatchNorm → 크기 · 채널이 바뀐 블록에서 **더하기가 가능하도록** 모양 맞춤
- **층 수 = 18**
conv1(1) + layer1~4(4 × 2 블록 × 2conv=16) + fc(1)
- **fc 입력이 512 인 이유**
avgpool 이 7 × 7 × 512 를 512 개 숫자로 요약

#06 모델 학습 함수 만들기 (코드 5-20)

```
def train_model(model, dataloaders, criterion, optimizer,
                device, num_epochs=13, is_train=True):
```

함수 흐름

준비

since = time.time() · 기록용 리스트 · best_acc = 0.0

에포크 반복

13 에포크 반복 · running_loss / corrects 초기화

배치 반복

데이터 GPU 로 이동 → 오른쪽 5 단계 → 손실 · 정답 수 누적

에포크 마무리

평균 손실 · 정확도 계산 · best_acc 갱신
torch.save 로 에포크마다 00.pth ~ 12.pth 저장

종료

학습 시간 · 최고 정확도 출력 · acc/loss 기록 반환

학습 한 스텝의 5 박자

1

zero_grad 기울기 0 으로 초기화

2

forward outputs = model(inputs)

3

loss criterion(outputs, labels)

4

backward loss.backward()

5

step optimizer.step()

+ 예측 : _, preds = torch.max(outputs, 1)

Q 왜 loss.item() * inputs.size(0)? → 배치 **평균** 손실을 **합계**로 되돌려 누적, 마지막에 전체 개수로 나눠야 정확한 평균

#07 옵티마이저 전달 & 모델 학습 (코드 5-21, 5-22)

5-21 학습할 파라미터만 옵티마이저에

```
params_to_update = []
for name, param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param)
        print("\t", name)          # fc.weight, fc.bias
optimizer = optim.Adam(params_to_update)
```

5-22 모델 학습 실행

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()
train_acc_hist, train_loss_hist = train_model(resnet18, train_loader,
                                              criterion, optimizer, device)
```

전달 인자 : **모델**, **학습 데이터**, **손실 함수**, **옵티마이저**, **장치 (CPU/GPU)**

학습 결과

0.937762

Best Acc (훈련) · 약 93.8%

6m 27s

Training complete

72%

첫 에포크부터 이미 높은 정확도

합성곱층이 이미 "**보는 법**"을 알고 있어서 fc 만 조금 학습해도 금방 성능이 나옴 = **전이 학습의 힘**

#08 테스트 데이터 호출 및 전처리 (코드 5-23)

```
test_path = '../chap05/data/catanddog/test'
transform = transforms.Compose([
    transforms.Resize(224),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
])
test_dataset = torchvision.datasets.ImageFolder(root=test_path, transform=transform)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=32,
                                           num_workers=1, shuffle=True)

print(len(test_dataset))
```

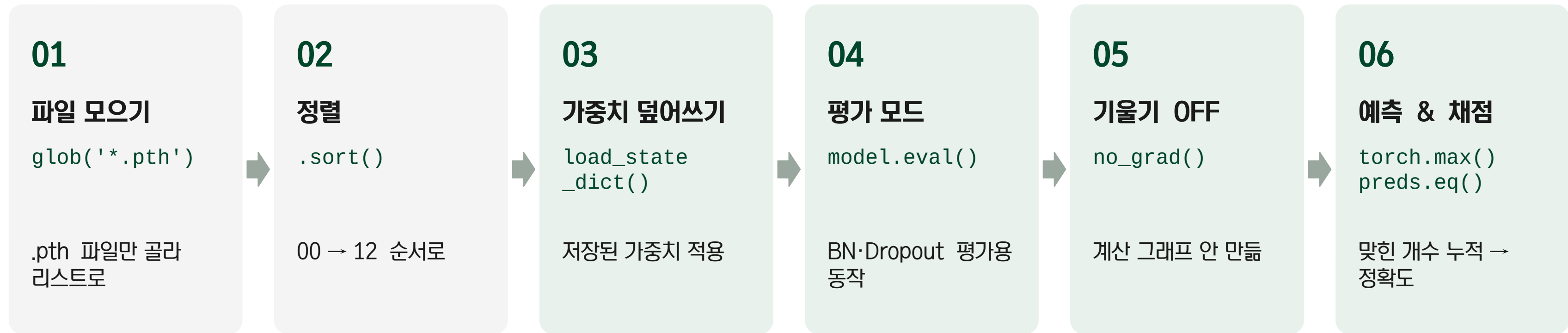
훈련용 vs 테스트용 전처리

	훈련 (5-13)	테스트 (5-23)
크기 조정	Resize([256, 256])	Resize(224) - 짧은 변을 224 로
자르기	RandomResizedCrop(224) - 랜덤	CenterCrop(224) - 가운데 고정
뒤집기	RandomHorizontalFlip()	없음
의도	데이터 확장으로 다양하게 학습	매번 같은 결과로 공정하게 평가

Tip 시험 볼 때 문제지를 매번 다르게
자르면 채점이 공정하지 않다
→
테스트 전처리에는 랜덤 요소를 뺀다

#09 테스트 데이터 평가 함수 (코드 5-24)

학습 중 저장한 **13 개 모델 (.pth)** 을 하나씩 불러와 테스트 데이터로 정확도를 측정



모델 13 개 각각 반복 (03~06)

glob — 원하는 파일만 골라 오기

- `'../chap05/data/catanddog/' + '*.pth'` → 이 경로에서 pth 확장자를 갖는 파일을 가져오라는 의미
- .pth 파일 = 훈련 때 **torch.save** 로 생성된 파일 (모델의 가중치가 저장됨)

#10 평가 함수 속 핵심 함수들 (코드 5-24)

torch.max(텐서 , 1) — 최댓값과 그 위치

- 최댓값과 **그 최댓값이 있는 index** 를 함께 반환
- _, preds → 최댓값은 버리고 **인덱스만 preds** 에 저장
- 두 번째 인자 1 = 행 안에서 **가로 방향**으로 찾기

예시	값
y_pred = [[0.2, 0.7, 0.8, 0.4]]	
최댓값	0.8
그 인덱스	2 ← preds 에 들어가는 값

preds.eq(labels).int().sum() — 맞힌 개수

부분	역할
preds.eq(labels)	자리마다 같으면 True, 다르면 False
.int()	True → 1, False → 0
.sum()	1 의 개수 = 예측과 정답이 일치한 개수

model.eval() vs torch.no_grad()

비슷해 보이지만 역할이 다름

	model.eval()	torch.no_grad()
바꾸는 것	층의 동작 방식	기울기 계산 여부
구체적으로	BatchNorm 은 훈련 때 모아 둔 평균 · 분산 사용 , Dropout 은 끄	계산 그래프를 만들지 않음 (autograd 사용 안 함)
효과	평가 결과가 매번 일정	메모리 절약 + 속도 향상
비유	" 시험 모드로 전환 "	" 채점만 하고 오답 노트는 안 씀 "

Point 평가할 때는 **둘 다 쓰는 게 정석**
훈련으로 돌아갈 땐 model.train()

#11 평가 결과와 그래프 (코드 5-25 ~ 5-27)

```
val_acc_hist = eval_model(resnet18, test_loader, device)
```

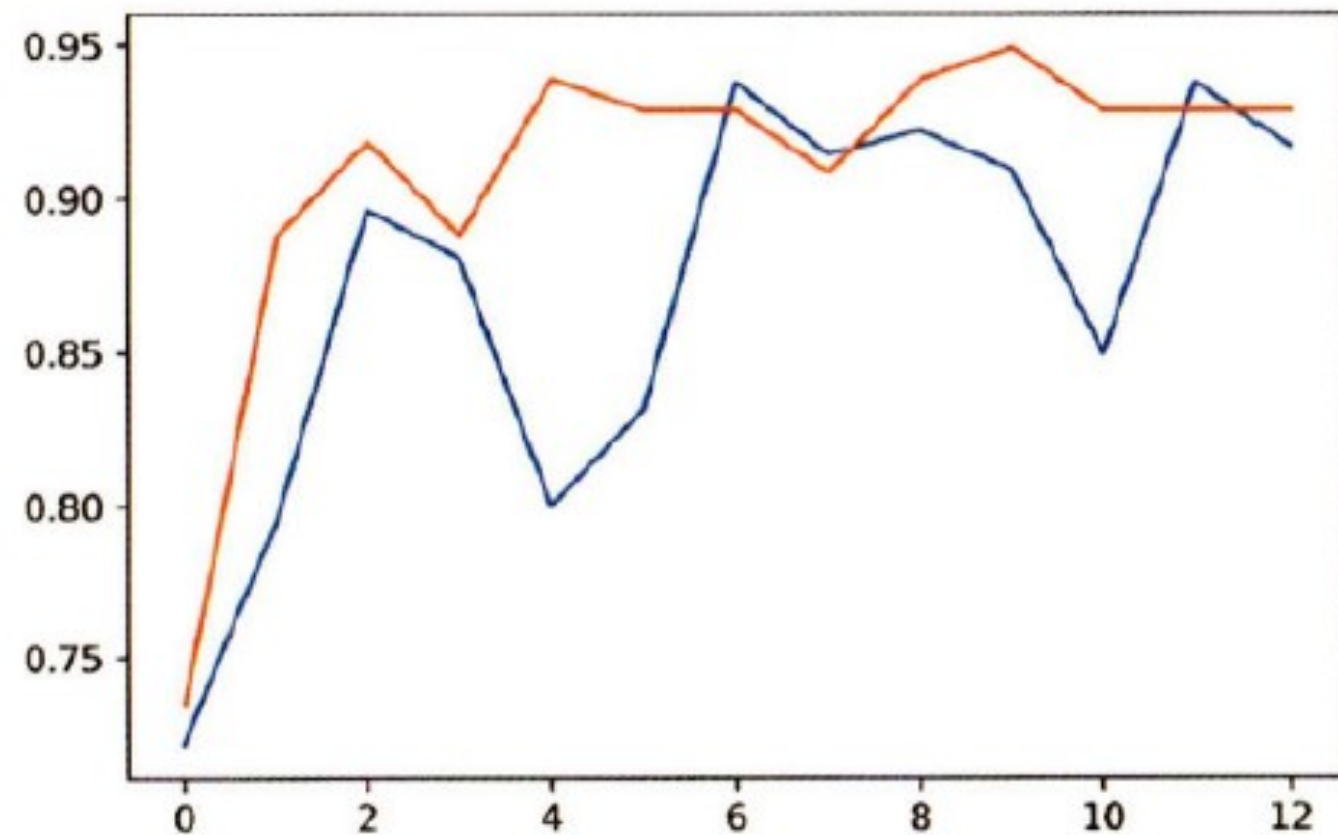
0.948980

Best Acc (테스트)

1m 52s

Validation complete

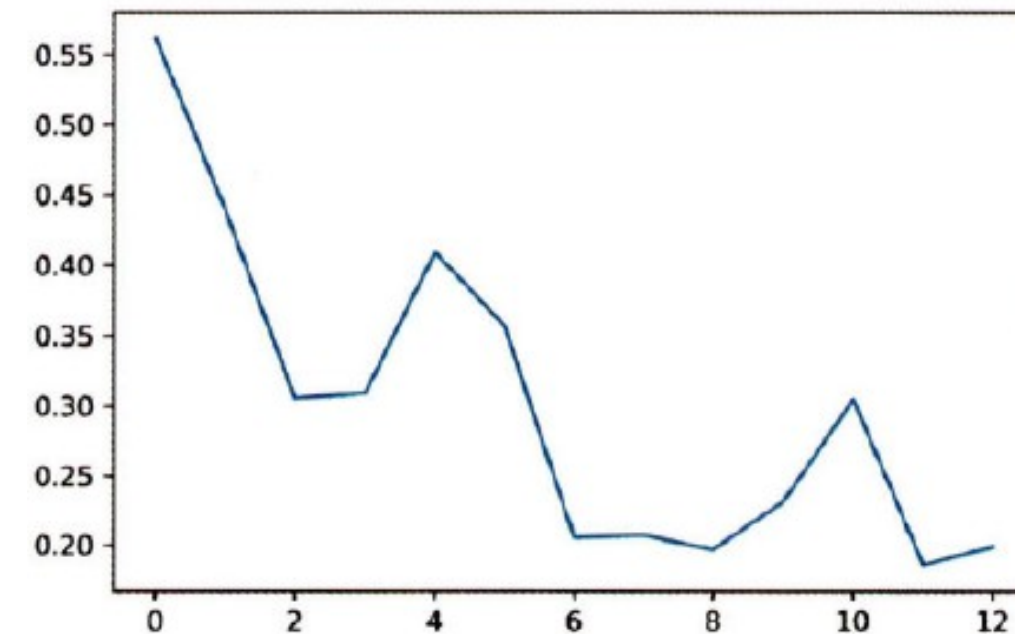
♥ 그림 5-37 훈련과 테스트 데이터에 대한 정확도



파랑 = 훈련

주황 = 테스트

♥ 그림 5-38 훈련 데이터에 대한 오차



- 두 곡선 모두 에포크가 진행될수록 **정확도가 높아짐**
- 오차 : 0.56 → 약 0.20 으로 **감소** → 학습이 잘 됐다고 볼 수 있음

Q 테스트가 훈련보다 정확도가 높은 이유 ? → 훈련은 Crop-Flip 으로 **일부러 어려운 사진**, 테스트는 **가운데를 깔끔하게 자른 사진**만 보기 때문 (데이터 수도 적음)

#12 예측 이미지 출력을 위한 전처리 함수 (코드 5-28)

정확도 · 오차 확인에 그치지 않고 , 실제로 잘 예측하는지 눈으로 보기 위해 텐서를 이미지로 되돌리는 함수

코드	하는 일	모양
<code>image = tensor.clone().detach().numpy()</code>	복사본 생성 + 기울기 추적 끊고 넘파이로	(3, 224, 224)
<code>image = image.transpose(1, 2, 0)</code>	matplotlib 형식으로 축 순서 변경 (C,H,W)→(H,W,C)	(224, 224, 3)
<code>image = image * 0.5 + 0.5</code>	정규화를 되돌려 원래 색으로 복원	그대로
<code>image = image.clip(0, 1)</code>	0~1 을 벗어난 값 잘라냄 (imshow 는 0~1 만 표시)	그대로

clone() vs detach()

구분	메모리	계산 그래프
<code>tensor.clone()</code>	새롭게 할당	계속 상주
<code>tensor.detach()</code>	공유해서 사용	상주하지 않음
<code>tensor.clone().detach()</code>	새롭게 할당	상주하지 않음

clip() — 범위 밖 값 잘라내기

`np.clip(exam, -0.5, 0.5)`

원래 값	-1.8	-0.7	0.0	0.8	1.9
clip 후	-0.5	-0.5	0.0	0.5	0.5

하한보다 작으면 하한, 상한보다 크면 상한, 사이 값은 그대로

#13 개와 고양이 예측 결과 출력 (코드 5-29)

```
classes = {0: 'cat', 1: 'dog'}
dataiter = iter(test_loader)
images, labels = dataiter.next()          # batch of 32
_, preds = torch.max(model(images), 1)
fig = plt.figure(figsize=(25, 4))
for idx in np.arange(20):
    ax = fig.add_subplot(2, 10, idx+1, xticks=[], yticks=[])
    plt.imshow(im_convert(images[idx]))    # from 5-28
    ax.set_title("{}({})".format(classes[preds[idx].item()],
                                   classes[labels[idx].item()]},
                color=("green" if preds[idx]==labels[idx] else "red"))
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

♥ 그림 5-43 개와 고양이에 대한 예측 결과



결과를 살펴보니 예측이 정확하지 않은 것을 확인할 수 있습니다. 훈련 데이터를 더 늘리고, 에포크 횟수도 늘려 보면 더 좋은 결과를 얻을 수 있을 것입니다.

포인트

- **add_subplot(2, 10, idx+1)** : 2 행 × 10 열 격자, 인덱스는 1 부터 → +1
- **xticks=[], yticks=[]** : 축 눈금 삭제 (사진엔 불필요)
- 제목 형식 "**예측 (정답)**" → 예: dog(cat) = 개라고 예측, 정답은 고양이
- 맞으면 **초록**, 틀리면 **빨강**
- **subplots_adjust** : Figure 안 서브플롯 위치 · 간격 조정

결론 예측이 정확하지 않은 것도 있음 → **훈련 데이터를 늘리고 에포크를 늘리면** 더 좋은 결과 기대

5.3.2 미세 조정 기법



#01 미세 조정 (Fine-tuning) 이란 ?

미세 조정 기법

: 특성 추출에서 더 나아가, 사전 훈련된 모델과 **합성곱층** · **데이터 분류기의 가중치를 업데이트**하여 훈련시키는 방식

- 특성 추출은 **목표 특성을 잘 추출했다는 전제** 하에 좋은 성능
- 특성이 잘못 추출됐다면 (예 : ImageNet 이미지 ≠ 전자상거래 물품 이미지) → 네트워크 가중치를 업데이트해 **특성을 다시 추출**
- 즉, 사전 학습된 모델을 **목적에 맞게 재학습**시키거나 **가중치의 일부를 재학습**
- 많은 연산량 필요 → CPU 보다 **GPU 사용 권장**

특성 추출 vs 미세 조정

	특성 추출 (5.3.1)	미세 조정 (5.3.2)
합성곱층	전부 고정	일부 또는 전체 재학습
데이터 분류기	학습	학습
전제	사전 학습된 특성이 내 데이터에도 잘 맞음	내 데이터 특징이 달라 특성을 다시 뽑아야 함
연산량	적음	많음 (GPU 권장)

#02 데이터셋 크기 x 유사성에 따른 4 가지 전략

미세 조정이 필요한지, 어디까지 재학습할지는 두 가지 기준으로 결정

유사성 큼 (ImageNet 과 비슷)

데이터셋
크

합성곱층 뒷부분 + 분류기 학습

데이터가 유사하므로 전체를 학습시키는 것보다 **강한 특징이 나타나는 뒷부분**과 분류기만 새로 학습해도 최적의 성능

데이터셋
작음

데이터 분류기만 학습

데이터가 적어 많은 계층에 미세 조정을 적용하면 **과적합** 가능 → 최종 분류기인 완전연결층에 대해서만 미세 조정

유사성 작음 (ImageNet 과 다름)

모델 전체 재학습

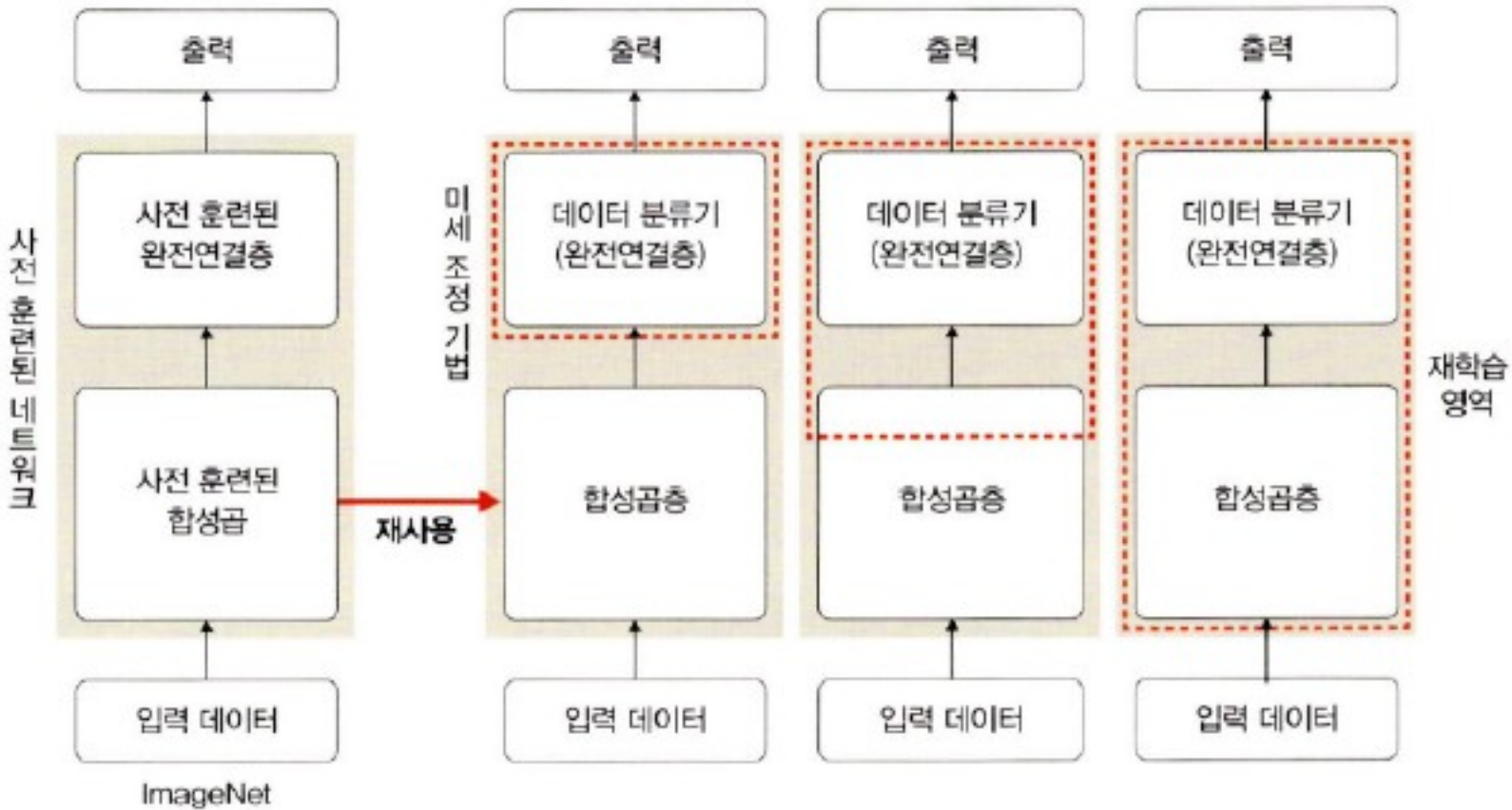
데이터셋이 크므로 **모델 전체를 재학습**시키는 것이 좋은 전략

합성곱층 일부 + 분류기 학습

데이터가 적어 일부 계층에만 적용해도 효과가 없을 수 있음 → 합성곱층 중 **어디까지 새로 학습시킬지** 적당히 설정

#03 재학습 영역 정리 & 주의점

▼ 그림 5-44 미세 조정 기법



재학습 영역 (그림 왼쪽부터)

모델	재학습 영역	대응 전략
사전 훈련 (ImageNet)	- (합성곱을 오른쪽으로 재사용)	-
①	데이터 분류기만	데이터 작음 + 유사성 큼
②	분류기 + 합성곱층 윗부분	데이터 큼 + 유사성 큼 데이터 작음 + 유사성 작음
③	분류기 + 합성곱층 전체	데이터 큼 + 유사성 작음

주의

- 파라미터 업데이트 과정에서 파라미터에 **큰 변화**를 주면 **과적합** 문제가 발생할 수 있음
- **정교하고 미세한** 파라미터 업데이트가 필요 (그래서 이름이 " 미세 " 조정 , 보통 작은 학습률 사용)

5.4 설명 가능한 CNN



#01 설명 가능한 CNN



정의

- 딥러닝 처리 결과를 **사람이 이해할 수 있는 방식으로** 제시하는 기술

필요성

- CNN의 예측 결과만으로는 내부 처리 과정을 이해하고 신뢰하기 어려움
- > **중간계층을 시각화**하면 입력 특징의 변화 확인 가능

시각화 방법

- **필터**에 대한 시각화 : 모델의 가중치 패턴 확인
- **특성 맵**에 대한 시각화 : 특정 입력에 대한 중간 계층의 반응 확인

#02 특성 맵 시각화

특성 맵(feature map)이란?

- 입력 이미지 또는 다른 특성 맵에 **필터를 적용한 결과**
- 활성화 맵이라고도 부름
- 각 필터는 이미지의 **서로 다른 특징에 반응**
- > 특성 맵을 통해 CNN이 이미지의 **어떤 특징을 학습하는지 확인 가능**

시각화의 목적

- 입력 이미지가 합성곱 계층을 거치면서 어떻게 변화하는지 확인
- 특성 맵에서 입력 특성을 감지하는 방법을 이해하는 데 도움

입력 이미지 -> Conv -> Feature Map

#03 라이브러리 호출

라이브러리 설치

> pip install pillow

```
import matplotlib.pyplot as plt
from PIL import Image
import cv2
import torch
import torch.nn.functional as F
import torch.nn as nn
from torchvision.transforms import ToTensor
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```


#04 네트워크 생성

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), ----- inplace=True는 기존의 데이터를 연산의
            nn.Dropout(0.3),                결괏값으로 대체하는 것을 의미
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
```

```
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

```
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

```
        )
        self.classifier = nn.Sequential(
            nn.Linear(512, 512, bias=False),
            nn.Dropout(0.5),
            nn.BatchNorm1d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )
```

```
def forward(self, x):
    x = self.features(x)
    x = x.view(-1, 512)
    x = self.classifier(x)
    return F.log_softmax(x) ----- ①
```


#05 네트워크 생성

네트워크 구성

- 합성곱층 13개, 완전연결층 2개
- 활성화 함수: ReLU

```
self.features = nn.Sequential(  
    nn.Conv2d(3, 64, kernel_size=3, bias=False),  
    nn.BatchNorm2d(64),  
    nn.ReLU(inplace=True), ----- inplace=True는 기존의 데이터를 연산의  
    nn.Dropout(0.3),                결괏값으로 대체하는 것을 의미  
    nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),  
    nn.BatchNorm2d(64),  
    nn.ReLU(inplace=True),  
    nn.MaxPool2d(kernel_size=2, stride=2),  
)
```

1) 특징 추출

- 여러 합성곱층을 순차적으로 구성
- Conv2d -> BatchNorm2d -> ReLU 구조 반복
- 일부 구간에 Dropout, MaxPool2d 적용

```
self.classifier = nn.Sequential(  
    nn.Linear(512, 512, bias=False),  
    nn.Dropout(0.5),  
    nn.BatchNorm1d(512),  
    nn.ReLU(inplace=True),  
    nn.Dropout(0.5),  
    nn.Linear(512, num_classes)  
)
```

2) 분류

- 추출된 특징 입력받아 최종 클래스 분류
- Linear -> Dropout -> BatchNorm1d -> ReLU
- 마지막 Linear 층에서 출력 차원을 num_classes로 출력

#06 네트워크 생성

```
def forward(self, x):  
    x = self.features(x)  
    x = x.view(-1, 512)  
    x = self.classifier(x)  
    return F.log_softmax(x) ..... ①
```

3) 순전파 과정

- 합성곱층 통과하며 **특징 추출**
- 완전연결층 통해 **분류**
- 최종 출력값 반환

로그 소프트맥스

- 소프트맥스 결과에 log 값을 취한 연산
- 신경망 마지막 출력을 **로그 확률**로 표현
- > 기본 소프트맥스의 **기울기 소멸 문제** 보완

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

$$\begin{aligned}\log\text{softmax} &= \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right) \\ &= x_i - \log\left(\sum_{j=1} e^{x_j}\right)\end{aligned}$$

#07 모델 객체화

```
model = XAI() ----- model이라는 이름의 객체를 생성  
model.to(device) ----- model을 장치(CPU 혹은 GPU)에 할당  
model.eval() ----- 테스트 데이터에 대한 모델 평가 용도로 사용
```

model.eval()

- 학습모드 -> 평가모드
- Dropout 비활성화 : 모든 뉴런 사용
- BatchNorm : 현재 배치 대신 학습 중 저장한 값으로 정규화
- > 평가 시 일관된 출력값을 얻기 위함

#08 특성 맵 확인 클래스 정의

- 특정 합성곱층의 출력값 확인
- 순전파 과정의 중간 결과를 가져와 시각화

```
class LayerActivations:
    features = []
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn) ----- ①

    def hook_fn(self, module, input, output):
        self.features = output.detach().numpy()

    def remove(self): ----- hook 삭제
        self.hook.remove()
```

코드 설명

- register_forward_hook(): 원하는 계층의 출력값 가져오기
- detach(): 출력값을 계산 그래프에서 분리하여 저장
- self.features: 가져온 특성 맵 저장
- remove(): 사용 끝난 hook 제거

#09 특성 맵 확인 클래스 정의

중간 변수와 Hook

```
import torch
x = torch.Tensor([0,1,2,3]).requires_grad_()
y = torch.Tensor([4,5,6,7]).requires_grad_()
w = torch.Tensor([1,2,3,4]).requires_grad_()
z = x + y;
o = w.matmul(z)
o.backward()
print(x.grad, y.grad, z.grad, w.grad, o.grad)

tensor([2., 3., 4., 5.]) tensor([2., 3., 4., 5.]) None tensor([ 4.,  6.,  8., 10.]) None
```

- 직접 생성한 텐서(x, y, w)는 기울기 확인 가능
- 중간 변수(z, o)는 기울기 저장 X
- > z.register_hook(hook_fn) 사용 시 중간 계층의 기울기 확인 가능

#10 이미지 호출

```
img = cv2.imread("../chap05/data/cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR) ----- ①
img = ToTensor()(img).unsqueeze(0) ----- ②
print(img.shape)
```

cv2.resize : 이미지 변경

- 이미지 크기를 100 x 100으로 변경
- interpolation: 보간법

보간법이란?

: 이미지 크기 변경할 때, 새롭게 필요한 픽셀 값을 추정해서 채우는 방법

- 기존 픽셀 데이터에 근사 함수 $f(x, y)$ 적용
- 확대 -> 새로 생기는 위치에 픽셀 값 매핑
- 축소 -> 여러 픽셀 정보를 압축하여 새로운 값으로 할당

#11 이미지 호출

```
img = cv2.imread("../chap05/data/cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR) ----- ①
img = ToTensor()(img).unsqueeze(0) ----- ②
print(img.shape)
```

ToTensor().unsqueeze(0) : Tensor 변환

- 이미지를 **Tensor**로 변환
- 맨 앞에 배치 차원 추가

unsqueeze()란?

: 텐서에 크기가 1인 새로운 차원을 추가하는 함수

```
import torch

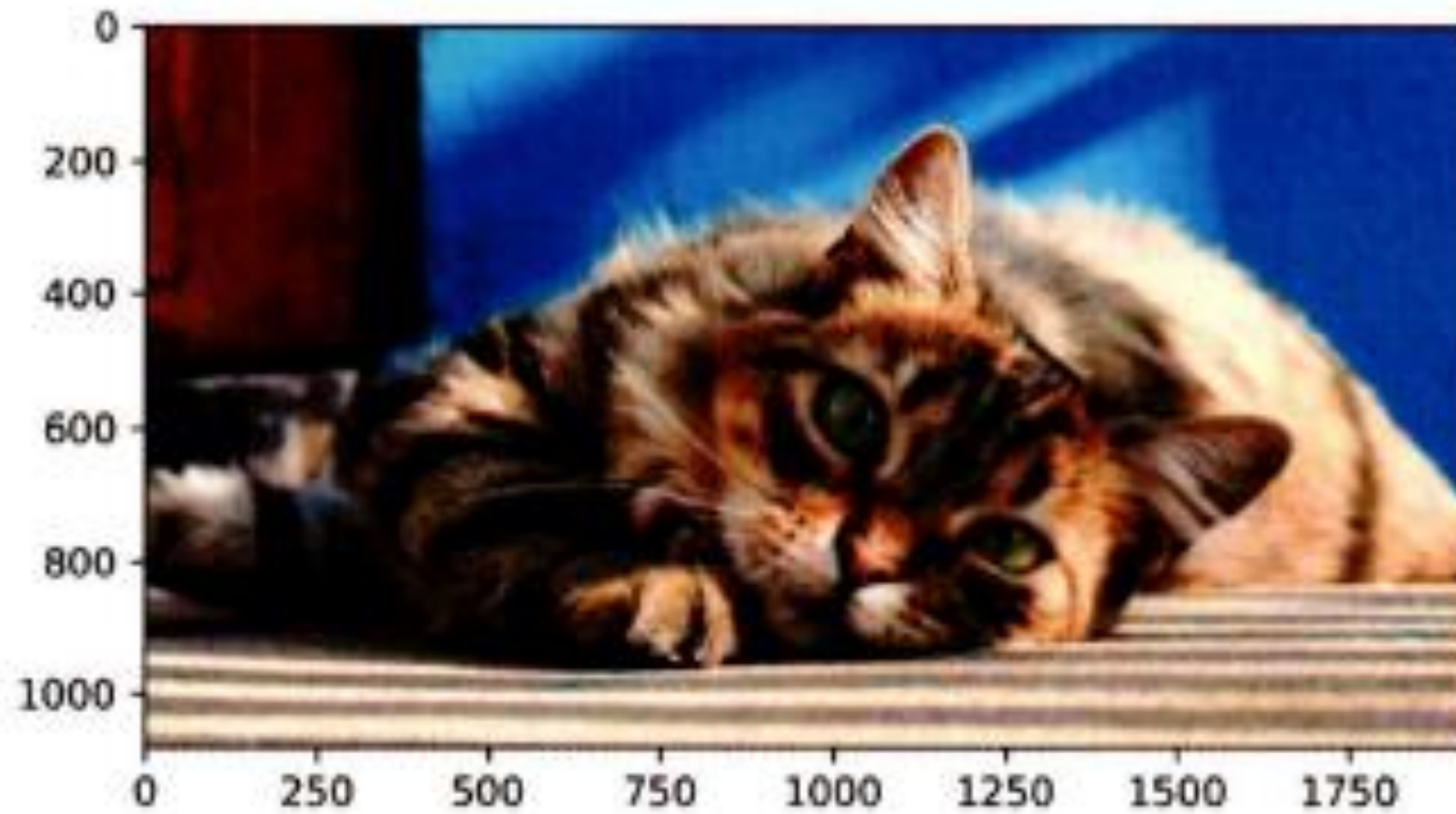
x1 = torch.rand(3, 10, 64)

x2 = x1.unsqueeze(dim=0) ----- [3, 10, 64] -> [1, 3, 10, 64]
print(x2.shape)
print('-----')
x3 = x1.unsqueeze(dim=1) ----- [3, 10, 64] -> [3, 1, 10, 64]
print(x3.shape)

torch.Size([1, 3, 10, 64])
-----
torch.Size([3, 1, 10, 64])
```

#12 이미지 호출

예제에서 사용할 이미지



#13 첫 번째 계층 특성 맵

```
result = LayerActivations(model.features, 0) ----- 0번째 Conv2d 특성 맵 확인
```

```
model(img)
```

```
activations = result.features
```

```
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

```
fig, axes = plt.subplots(4, 4)
```

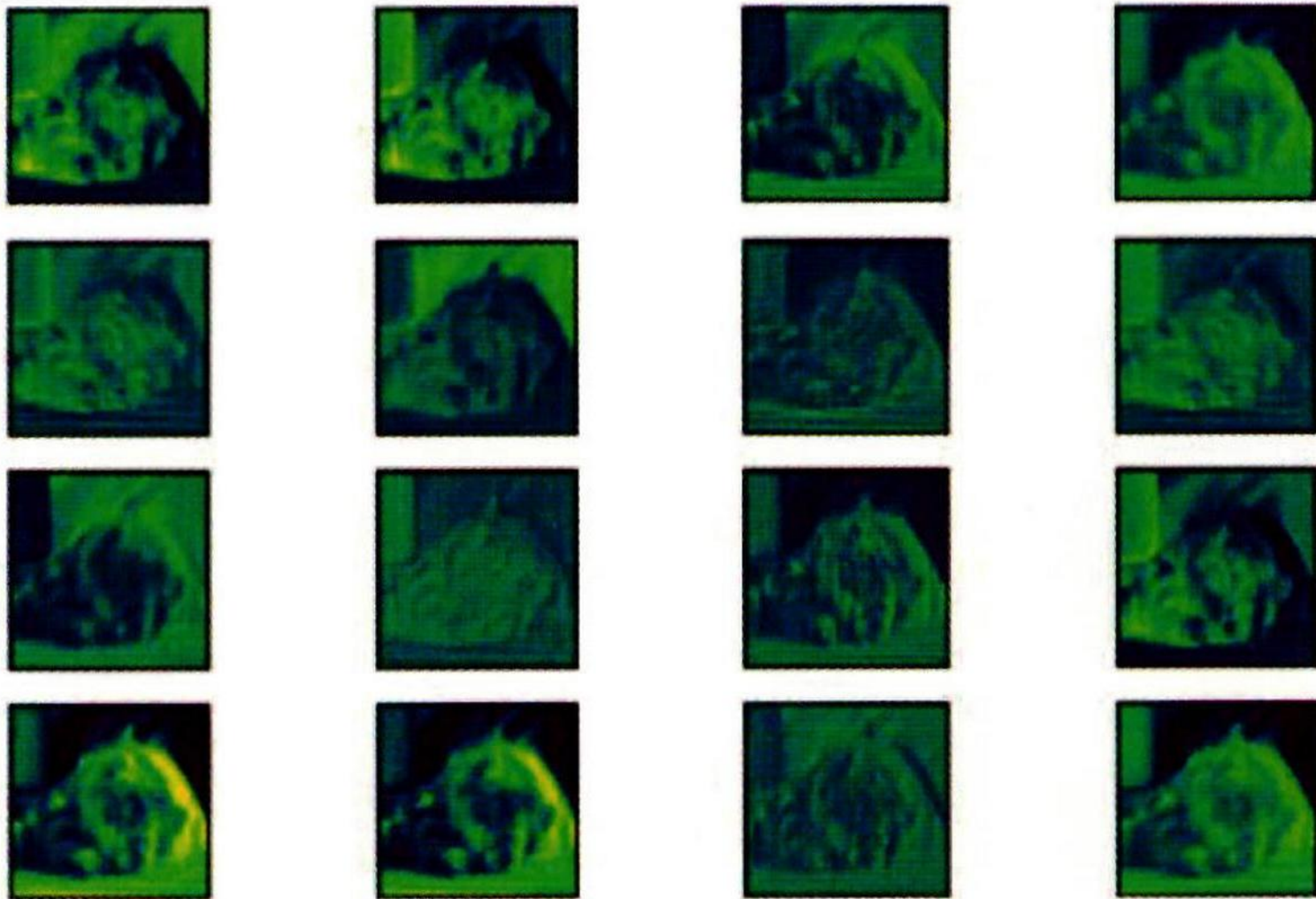
```
fig = plt.figure(figsize=(12,8))
```

```
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
```


#14 첫 번째 계층 특성 맵

```
result = LayerActivations(model.features, 0) ----- 0번째 Conv2d 특성 맵 확인
```

```
model(img)  
activations = result.features
```



결과

- 입력층과 가까운 계층
- 입력 이미지의 형태가 많이 유지됨

#15 20번째 계층 특성 맵

```
result = LayerActivations(model.features, 20) ----- 20번째 Conv2d 특성 맵 확인

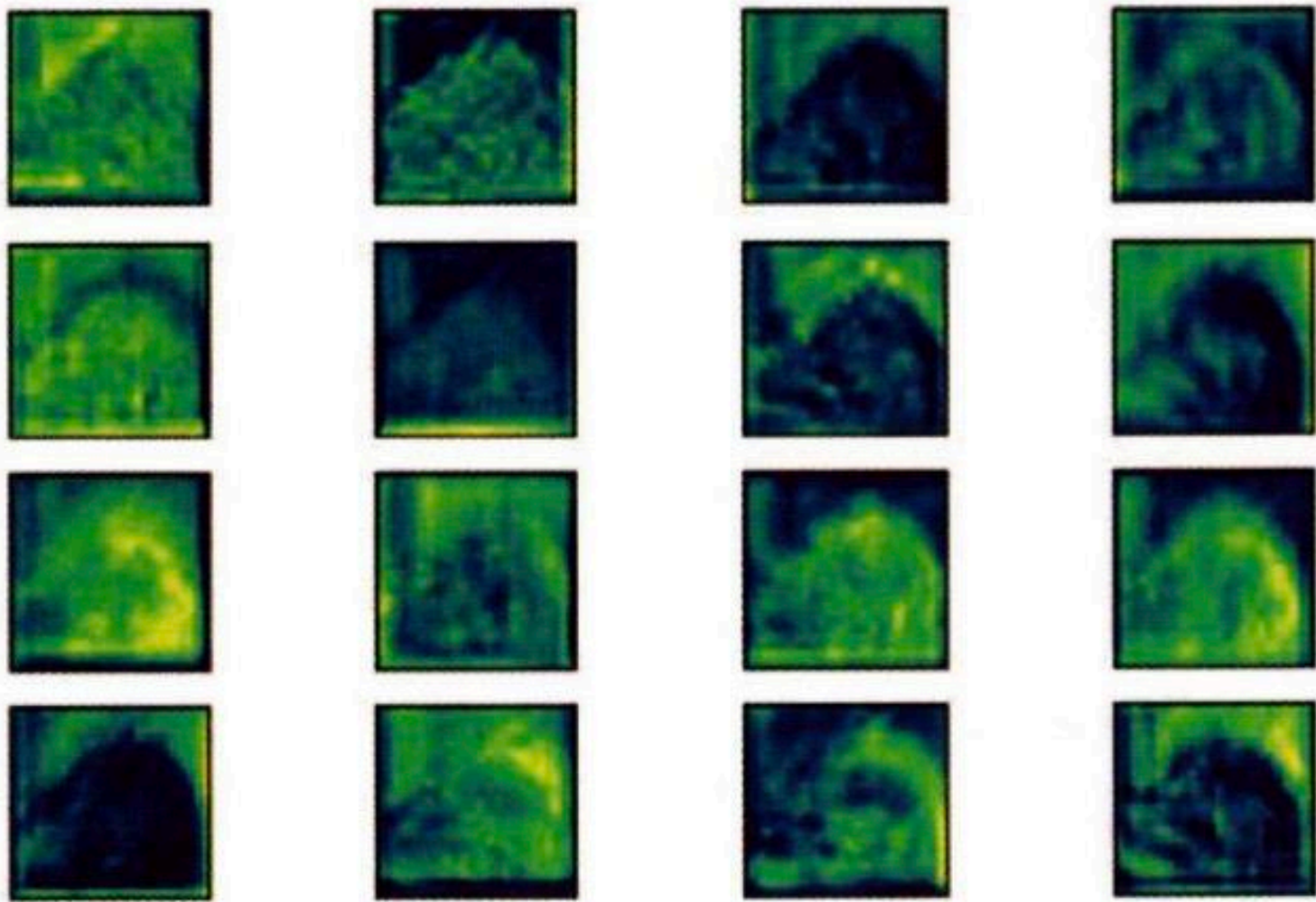
model(img)
activations = result.features

fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

#16 20번째 계층 특성 맵

```
result = LayerActivations(model.features, 20) ----- 20번째 Conv2d 특성 맵 확인
```

```
model(img)  
activations = result.features
```



결과

- 이미지 형태가 점차 변화
- 특징이 더 강조됨

#17 40번째 계층 특성 맵

```
result = LayerActivations(model.features, 40) ----- 40번째 Conv2d 특성 맵 확인

model(img)
activations = result.features

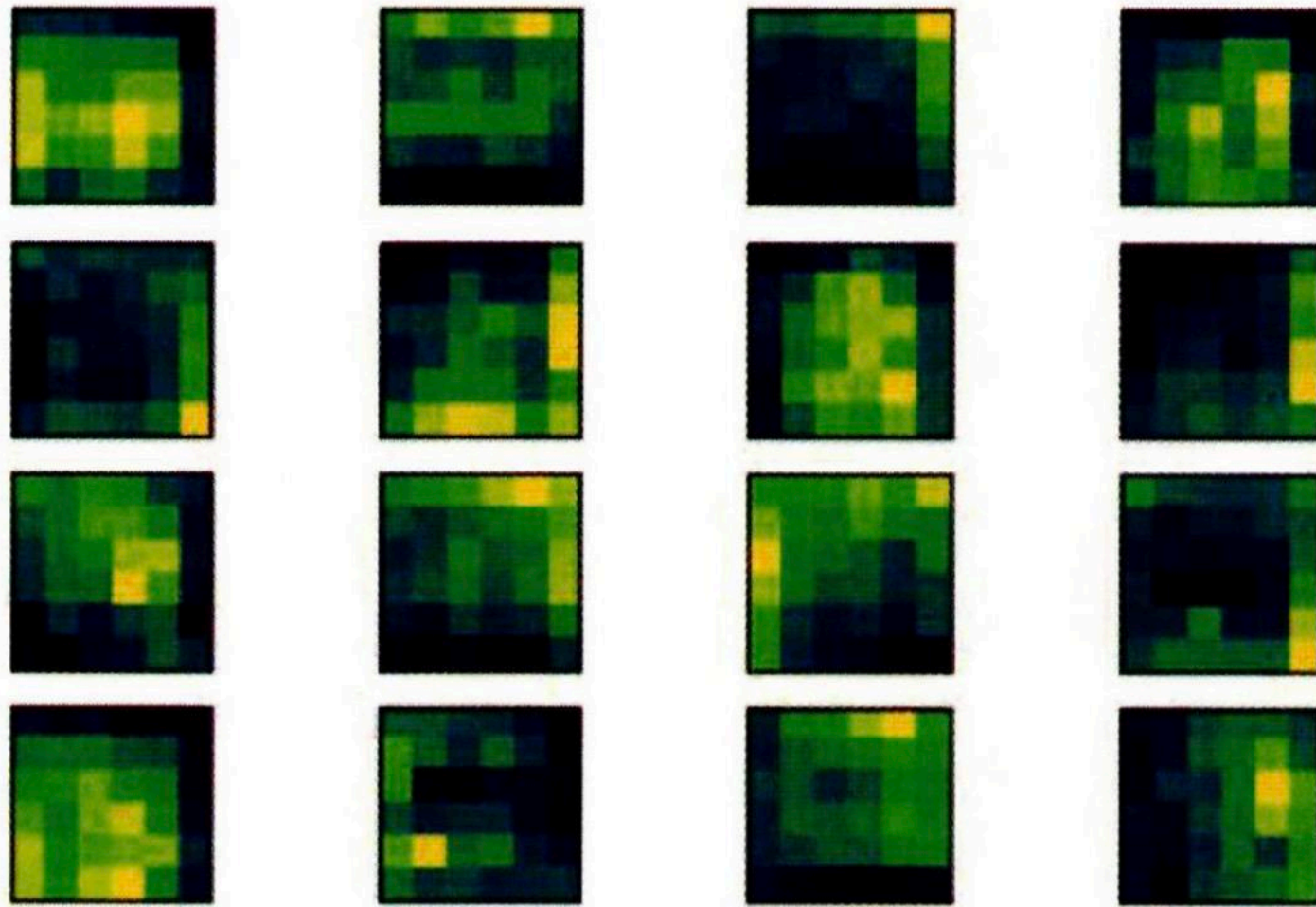
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)
for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))
```


#18 40번째 계층 특성 맵

```
result = LayerActivations(model.features, 40) ----- 40번째 Conv2d 특성 맵 확인
```

```
model(img)
```

```
activations = result.features
```

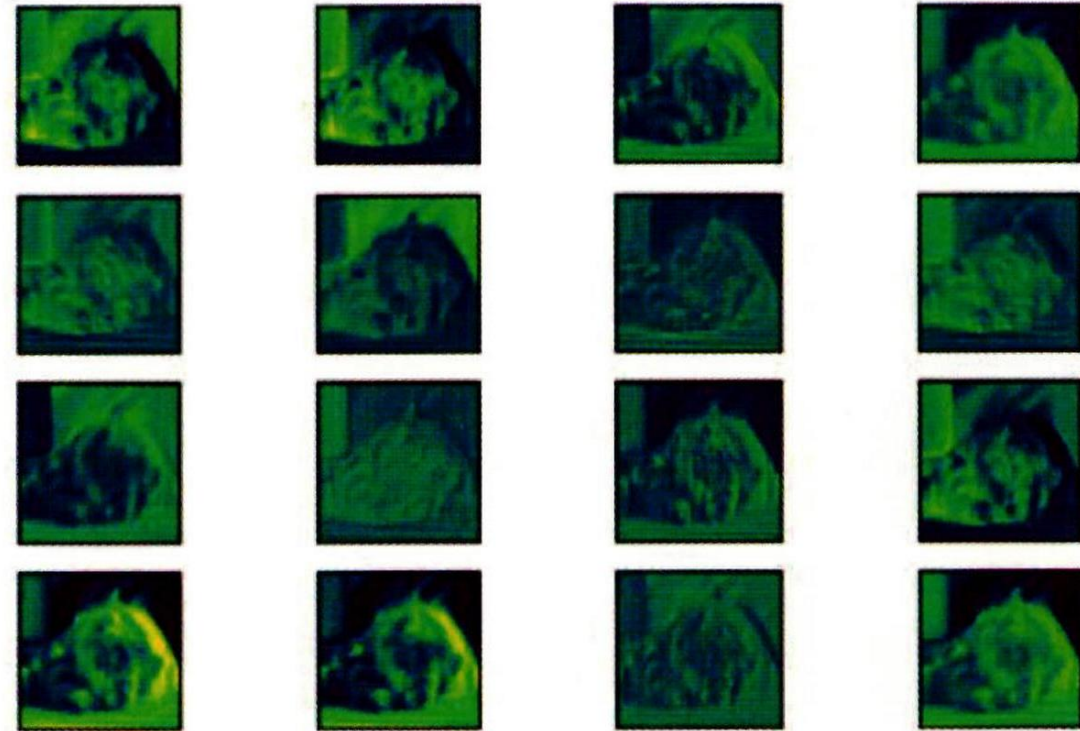


결과

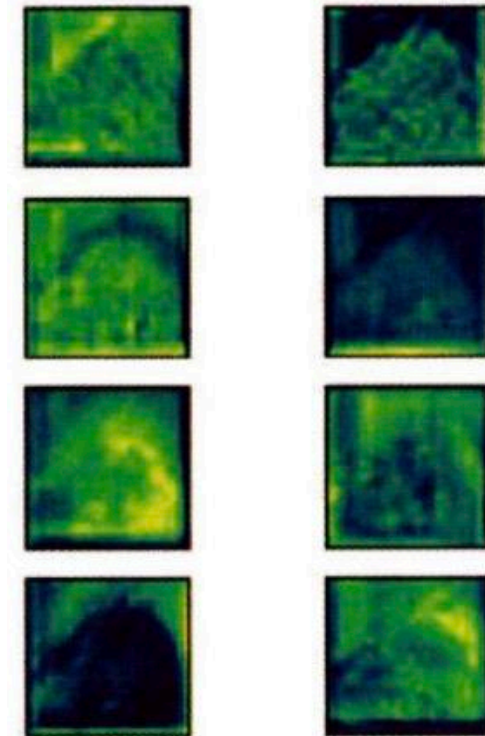
- 원래 형태를 거의 확인하기 어려움
- 이미지 특징 중심으로 전달

#19 계층별 비교

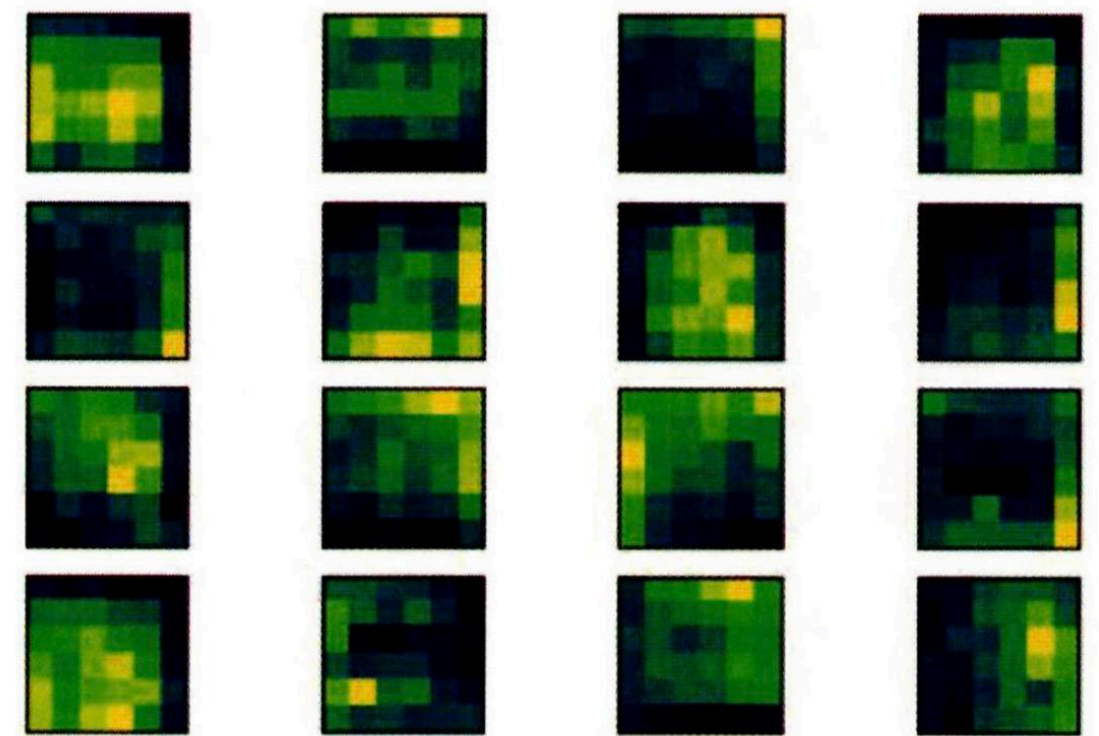
첫 번째 계층(0)



20번째 계층



40번째 계층



출력 결과 해석

- 출력층에 가까워질수록 원래 이미지 형태는 사라짐
- 학습된 특징이 중심이 되어 전달됨

#20 설명 가능한 CNN

신뢰성 문제

- 최근 딥러닝 결과에 대한 **신뢰성 문제** 제기되며, 설명 가능한 인공지능(XAI) 이슈가 됨
 - CNN은 필터와 특성 맵을 시각화하여 내부 처리 과정을 확인
- > 모델의 판단 근거를 이해하고 결과의 신뢰성 확보에 활용

5.5 그래프 합성곱 네트워크



#01 그래프란?

그래프

: 방향성이 있거나(directed) 없는(undirected) 에지로

연결된 노드의 집합

- 노드, 에지는 문제에 대한 지식, 직관 바탕으로 구성

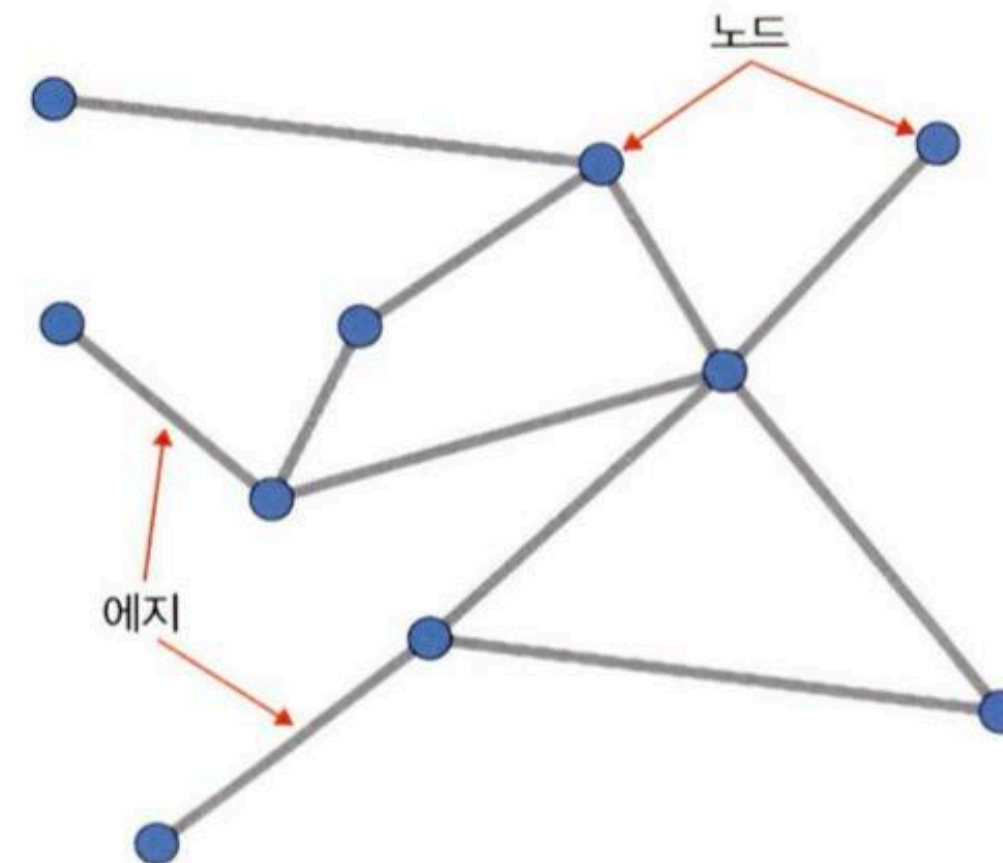
구성 요소

- 노드(node, vertex)

: 그래프에서 대상을 나타내는 점 = 원소

- 에지(edge)

: 두 노드를 연결한 선 = 결합 방법

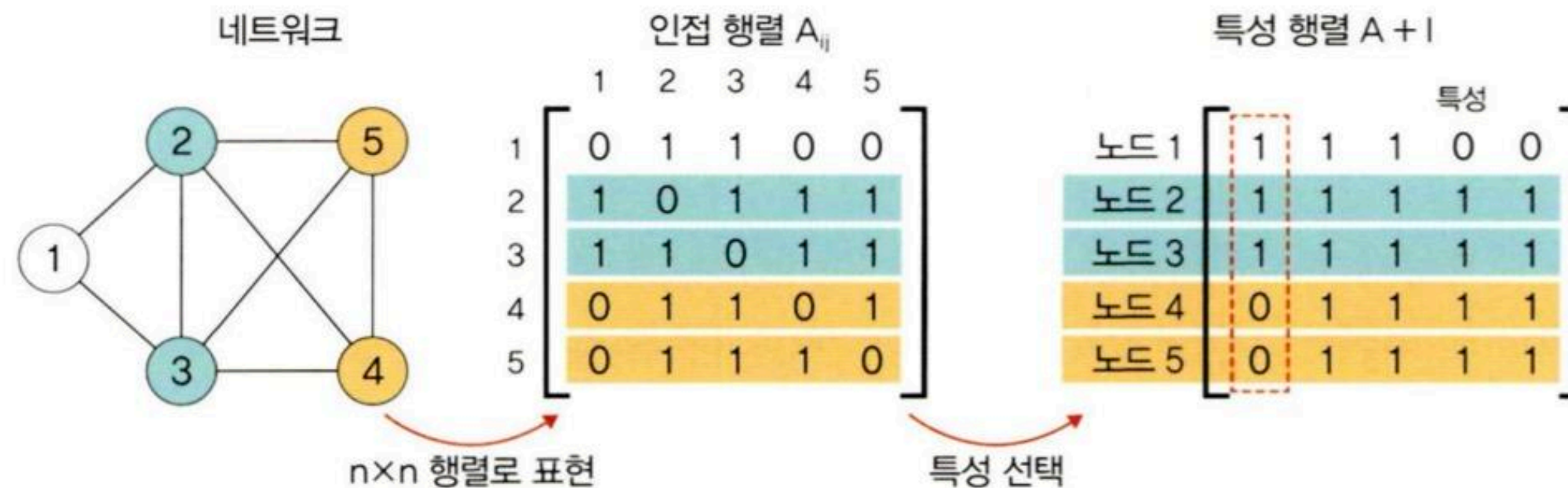


#02 그래프 신경망

그래프 신경망(Graph Neural Network, GNN)이란?

: 그래프 구조에서 사용하는 신경망

- 그래프 데이터를 컴퓨터가 이해할 수 있도록 행렬 형태로 표현



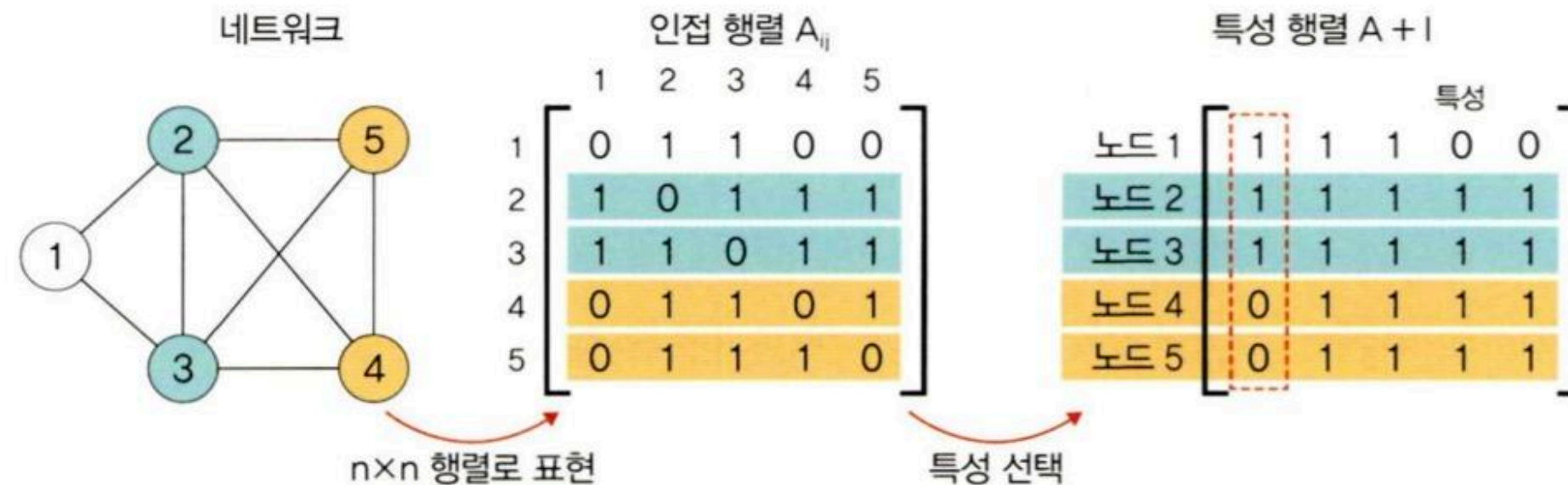
#03 그래프 신경망

1단계 : 인접 행렬(adjacency matrix)

- 노드 n 개를 $n \times n$ 행렬로 표현
- 인접 행렬 내의 값 : A_{ij} 에 노드 i 와 j 의 관련성 여부
- > 그래프의 연결 관계 표현

2단계 : 특성 행렬(feature matrix)

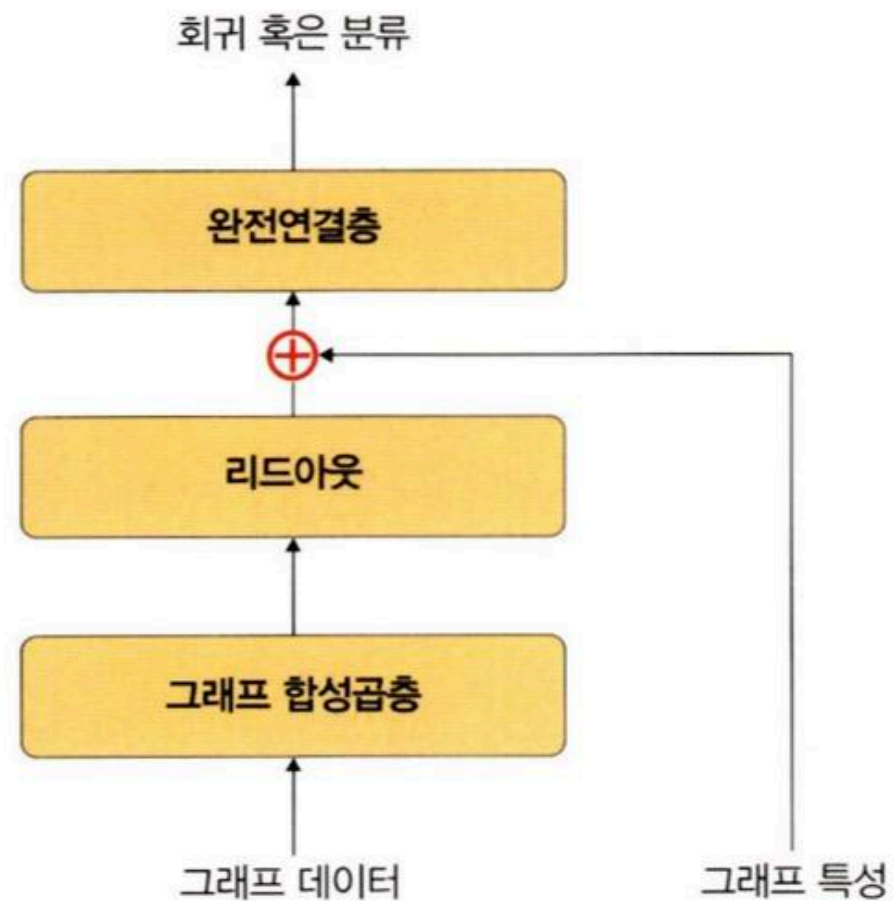
- 각 입력 데이터에서 이용할 특성 선택
- 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미
- 교재 예시: 인접 행렬 A 에 단위 행렬 I 를 더한 $A+I$ 사용



#04 그래프 합성곱 네트워크

GCN(Graph Convolutional Network)

: 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘



그래프 합성곱층

- 그래프 형태의 데이터를 **행렬 형태로 변환**
- > 일반적인 딥러닝 알고리즘을 적용할 수 있도록 함

리드아웃(readout)

- 노드의 특성 행렬을 **하나의 벡터**로 변환
- 예시: 전체 노드 특성 벡터의 평균
- **그래프 전체**를 표현하는 벡터 생성

#05 그래프 합성곱 네트워크

GCN의 활용 분야

- SNS에서의 관계 네트워크
- 학술 연구에서의 인용 네트워크
- 3D Mesh : 3D 물체의 표면을 노드와 에지의 연결 구조로 표현한 데이터

THANK YOU





4주차 질문

5장: 합성곱 신경망

질문1: 객채원

Q. 특성맵에 의한 시각화와 필터에 의한 시각화 차이는 무엇이며, 출력층에 가까운 계층들의 특징을 더 잘 시각화하는 방법은 무엇인가요?

질문2: 김기연

Q. 5.1.2에서 합성곱층의 출력 데이터 D2는 필터 개수(K)와 같다고 배웠습니다. 그런데 코드 5-9를 보면 layer1에서는 out_channels=32였다가 layer2에서는 out_channels=64로, 풀링을 거쳐 공간 크기(가로 × 세로)는 계속 줄어드는데 채널 수는 오히려 늘어나는 구조입니다. 여기서 공간 정보는 압축하면서 채널(특성 종류)은 왜 늘리는지 궁금합니다.

Q.

논의 질문) 코드 5-24의 ②번 설명에서는 `torch.max()`를 적용하면 최댓값의 인덱스가 반환된다고 설명하고 있습니다. 따라서 `preds`에는 이미 각 데이터에 대한 클래스 인덱스(예: 0 또는 1)가 저장됩니다.

그런데 이후 코드에서는 `preds`를 다시 0.5를 기준으로 나누어 1과 0으로 변환하고 있습니다.

```
preds[preds >= 0.5] = 1  
preds[preds < 0.5] = 0
```

이 과정이 실제로 필요한 과정일까요? `torch.max()`가 반환하는 값의 의미를 바탕으로 해당 코드의 필요성을 검토해 본 뒤, 불필요한 부분이 있다면 코드를 어떻게 수정할 수 있을지 이야기해보고 싶습니다.